

Para esta práctica usaremos 4 bases de datos en total:

Una Base Principal llamada “Farmacias_Ecuador”. Esta solo nos servirá como punto de partida para crear a las otras tres.

Tres Bases Nodos (quito, manta, guayaquil) que simularán nuestras sucursales físicas.

Creamos los nodos dentro de la base de datos “Farmacias_Ecuador”

```
CREATE DATABASE quito;
```

```
CREATE DATABASE manta;
```

```
CREATE DATABASE guayaquil;
```

Nos ubicamos dentro del nodo Manta, se crea la tabla ventas_medicamentos y se insertan los datos

```
CREATE TABLE ventas_medicamentos (  
    id SERIAL PRIMARY KEY,  
    medicamento TEXT,  
    cantidad INT,  
    precio NUMERIC,  
    ciudad TEXT  
);
```

```
INSERT INTO ventas_medicamentos VALUES  
(DEFAULT, 'Paracetamol 500mg', 15, 0.10, 'Manta'),  
(DEFAULT, 'Ibuprofeno 400mg', 22, 0.25, 'Manta');
```

Dentro del nodo Guayaquil se crea la tabla ventas_medicamentos y se insertan los datos

```
CREATE TABLE ventas_medicamentos (  
    id SERIAL PRIMARY KEY,  
    medicamento TEXT,  
    cantidad INT,
```

```
    precio NUMERIC,  
    ciudad TEXT  
);
```

```
INSERT INTO ventas_medicamentos VALUES  
(DEFAULT, 'Amoxicilina 500mg', 8, 0.60, 'Guayaquil'),  
(DEFAULT, 'Loratadina 10mg', 40, 0.15, 'Guayaquil');
```

Nos ubicamos en el nodo Quito y nuevamente creamos la tabla ventas_medicamentos y se hace la inserción de datos

```
CREATE TABLE ventas_medicamentos (  
    id SERIAL PRIMARY KEY,  
    medicamento TEXT,  
    cantidad INT,  
    precio NUMERIC,  
    ciudad TEXT  
);
```

```
INSERT INTO ventas_medicamentos VALUES  
(DEFAULT, 'Omeprazol 20mg', 30, 0.30, 'Quito'),  
(DEFAULT, 'Vitamina C 1g', 12, 0.80, 'Quito');
```

Configuración del Nodo Coordinador (Local - Quito) e Integración FDW

(FDW son las siglas de Foreign Data Wrapper (que en español vendría siendo algo como "Adaptador de Datos Externos") es básicamente un “puente” entre los nodos

```
CREATE EXTENSION postgres_fdw;
```

```
CREATE SERVER servidor_manta FOREIGN DATA WRAPPER postgres_fdw  
OPTIONS (dbname 'manta', host 'localhost', port '5432');
```

```
CREATE SERVER servidor_guayaquil FOREIGN DATA WRAPPER postgres_fdw  
OPTIONS (dbname 'guayaquil', host 'localhost', port '5432');
```

OJO cambiar '123' por su contraseña normal

```
CREATE USER MAPPING FOR postgres SERVER servidor_manta OPTIONS (user  
'postgres', password '123');
```

```
CREATE USER MAPPING FOR postgres SERVER servidor_guayaquil OPTIONS  
(user 'postgres', password '123');
```

```
CREATE SCHEMA node_manta; IMPORT FOREIGN SCHEMA public FROM  
SERVER servidor_manta INTO node_manta;
```

```
CREATE SCHEMA node_guayaquil; IMPORT FOREIGN SCHEMA public FROM  
SERVER servidor_guayaquil INTO node_guayaquil;
```

Se crea la tabla de catálogo local para cruces distribuidos

```
CREATE TABLE laboratorios_medicos (  
medicamento TEXT PRIMARY KEY,  
laboratorio TEXT, requiere_receta TEXT  
);
```

Inserción de datos

```
INSERT INTO laboratorios_medicos VALUES  
( 'Paracetamol 500mg', 'Bayer', 'NO'),  
( 'Ibuprofeno 400mg', 'Genfar', 'NO'),  
( 'Amoxicilina 500mg', 'Bago', 'SI'),  
( 'Loratadina 10mg', 'Mk', 'NO'),  
( 'Omeprazol 20mg', 'Genfar', 'NO'),  
( 'Vitamina C 1g', 'Bayer', 'NO');
```

OJO PARA VER LOS DATOS DEBEN CORRER EL CÓDIGO SIN “EXPLAIN ANALYZE”

Consulta 1: Ejecución Local

```
EXPLAIN ANALYZE SELECT * FROM ventas_medamentos;
```

Esta consulta trae únicamente los registros de la farmacia en Quito, demuestra la velocidad nativa. Al no requerir red, el tiempo de respuesta es de microsegundos, demostrando que la fragmentación permite a cada sucursal operar sin depender de las demás.

Consulta 2: Consulta Global Distribuida

EXPLAIN ANALYZE

SELECT * FROM ventas_medicamentos

UNION ALL

SELECT * FROM node_manta.ventas_medicamentos

UNION ALL

SELECT * FROM node_guayaquil.ventas_medicamentos;

Esta consulta consolida el inventario de todas las sucursales a nivel nacional.

Usando un operador Append une la tabla local con dos Foreign Scan (peticiones a los servidores remotos a través del FDW).

Evidencia el costo de red. Esta es la base teórica del procesamiento distribuido, donde se observa cómo el *Execution Time* sube debido a la latencia de ir a traer datos a otras bases lógicas

Optimización Global (Pushdown de Filtros)

EXPLAIN ANALYZE

SELECT * FROM ventas_medicamentos WHERE ciudad = 'Quito'

UNION ALL

SELECT * FROM node_manta.ventas_medicamentos WHERE ciudad = 'Manta'

UNION ALL

SELECT * FROM node_guayaquil.ventas_medicamentos WHERE ciudad = 'Guayaquil';

Busca de manera global, pero filtrando las ventas directamente por su ciudad de origen.

El optimizador de PostgreSQL empuja (*pushdown*) la condición WHERE hacia cada nodo respectivo para que filtre los datos antes de enviarlos por la red.

Es una técnica clave de optimización. Reduce drásticamente el tamaño del paquete de datos que viaja, mejorando los tiempos y evitando cuellos de botella en el ancho de banda.

Join Distribuido

EXPLAIN ANALYZE

SELECT l.laboratorio, SUM(v.cantidad) AS cajas_vendidas

FROM (

 SELECT * FROM ventas_medicamentos

 UNION ALL

 SELECT * FROM node_manta.ventas_medicamentos

 UNION ALL

 SELECT * FROM node_guayaquil.ventas_medicamentos

) v

JOIN laboratorios_medicos l ON v.medimento = l.medimento

GROUP BY l.laboratorio;

Genera un reporte gerencial agrupando el total de cajas vendidas en todo el país por cada laboratorio médico.

Junta los fragmentos remotos y locales en memoria, y ejecuta un algoritmo de cruce (Hash Join) contra la tabla estática laboratorios_medicos alojada solo en Quito.

Demuestra que no es necesario duplicar tablas maestras o catálogos en todos los nodos. Permite realizar cruces analíticos complejos manteniendo una única fuente de la verdad para metadatos como proveedores o precios.

